# CS301: CS301 Lecture Slides: Concurrency & Deadlocks

## CS301 Lecture 12: Concurrency & Synchronization

Instructor: Department of Computer Science

Welcome to Module 3: Concurrency Control and Synchronization.

Today we cover mutual exclusion primitives, critical section criteria,

and system-level deadlock emergence in modern multi-threaded operating systems.

# CS301: CS301 Lecture Slides: Concurrency & Deadlocks

## Slide 2: Critical Section Problem Criteria

Any valid solution to the critical section problem must satisfy three strict requirements:

1. Mutual Exclusion: If process Pi is executing in its critical section, no other processes can enter.
2. Progress: If no process is executing in its critical section and some wish to enter, only processes not in remainder section can participate in the selection.
3. Bounded Waiting: There must be a bound on the number of times other processes are allowed to enter their critical sections after a process has made a request to enter.

# CS301: CS301 Lecture Slides: Concurrency & Deadlocks

## Slide 3: Summary of the Four Coffman Conditions

Remember the acronym M-H-N-C:

- Mutual Exclusion (exclusive ownership)
- Hold and Wait (holding while requesting)
- No Preemption (cannot steal)
- Circular Wait (chain of waiting dependencies)

ALL 4 conditions MUST be simultaneously true for deadlock to manifest.

# CS301: CS301 Lecture Slides: Concurrency & Deadlocks

## Slide 4: Real-world Deadlock in Multi-tier Storage

Deadlocks are not merely theoretical; they routinely occur in database transaction managers and distributed storage arrays with two-phase locking protocols.

Graph cycle detection is actively implemented in PostgreSQL and MySQL InnoDB engines.

# CS301: CS301 Lecture Slides: Concurrency & Deadlocks

## Slide 5: Starvation Mitigation via Aging

**"Aging gradually increases the priority of processes that wait in the system for a long time."**

To resolve starvation caused by priority scheduling algorithms, operating systems implement aging. Over time, an aging priority scheduler ensures that even the lowest-priority process will eventually become the highest-priority runnable process and gain access to the required resource.